Methods have names, as do many other constructs in Ruby. Names in Ruby have special rules—rules that you may not have seen if you come to Ruby from another language.

Ruby Names

Local variables, method parameters, and method names should all start with a lowercase letter or with an underscore: `order`, `line_item`, and `xr2000` are all valid. Instance variables begin with an at (@) sign—for example, `@quantity` and `@product_id`. The Ruby convention is to use underscores to separate words in a multiword method or variable name (so `line_item` is preferable to `lineItem`).

Class names, module names, and constants must start with an uppercase letter. By convention they use capitalization, rather than underscores, to distinguish the start of words within the name. Class names look like `Object`, `PurchaseOrder`, and `LineItem`.

Rails uses *symbols* to identify things. In particular, it uses them as keys when naming method parameters and looking things up in hashes. Here's an example:

```
redirect_to :action => "edit", :id => params[:id]
```

As you can see, a symbol looks like a variable name, but it's prefixed with a colon. Examples of symbols include `:action`, `:line_items`, and `:id`. You can think of symbols as string literals magically made into constants. Alternatively, you can consider the colon to mean *thing named*, so `:id` is the thing named `id`.

Now that we've used a few methods, let's move on to how they're defined.

Methods

Let's write a method that returns a cheery, personalized greeting. We'll invoke that method a couple of times:

```
def say_goodnight(name)
  result = 'Good night, ' + name
  return result
end
```